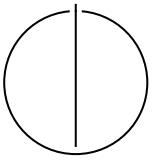
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Rodrigo Felix Forno



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

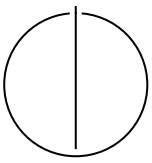
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Vergleichende Analyse von weit verbreiteten Zipfian Distribution Implementierungen

Author:	Rodrigo Felix Forno
Examiner:	Prof. Dr. Viktor Leis
Supervisor:	MSc. Bohyun Lee, MSc. Gabriel Haas
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Rodrigo Felix Forno

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section 1
1.1.1 Subsection 1

2 Approach **3**
2.1 Research Approach 3
2.2 Common Frameworks 4
2.2.1 Yahoo! Cloud Serving Benchmark 5
2.2.2 Flexible I/O Tester 6
2.2.3 Sysbench 7
2.2.4 OLTP-Bench 7

Abbreviations **8**

List of Figures **9**

List of Tables **10**

Bibliography **11**

1 Introduction

1.1 Section

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}`, `\ac{TUM}` ⇒ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 1.1, Figure 1.1, Figure 1.2, Figure 1.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

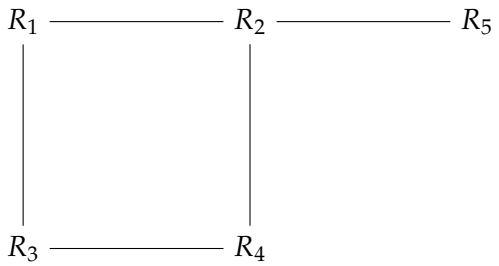


Figure 1.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

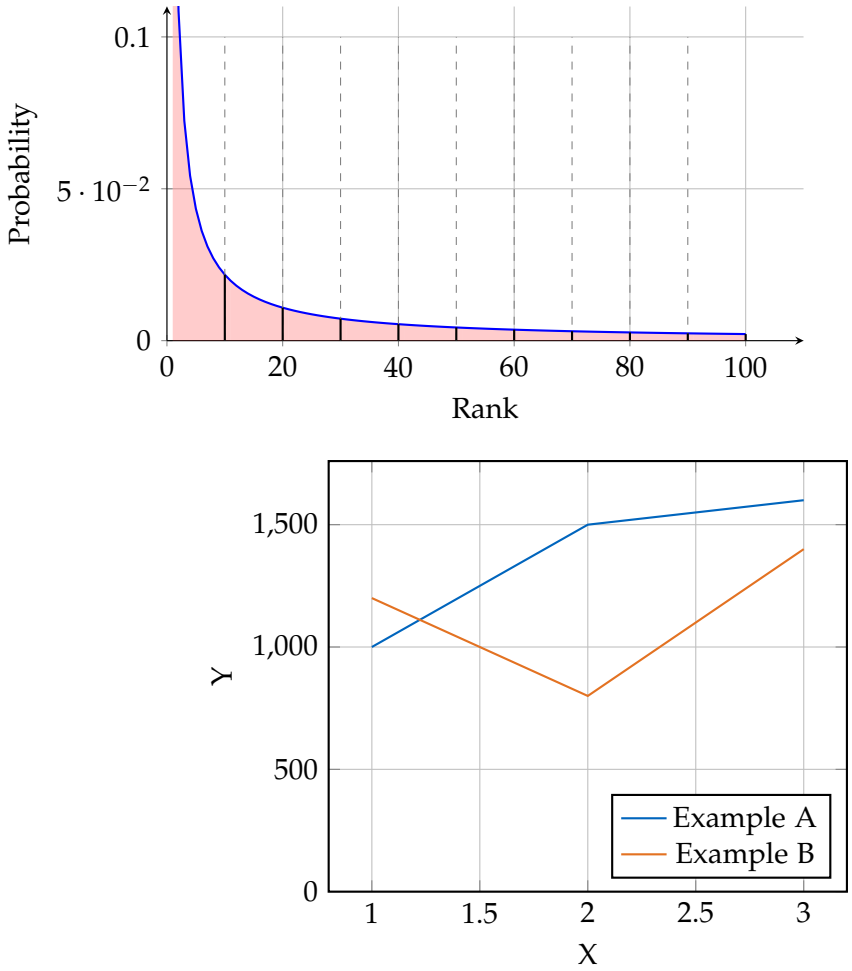


Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Approach

The goal of this thesis is to identify the most widely used frameworks for generating zipfian synthetic data in the systems programming and database community and make a comparative evaluation. The first step required to achieve this is a systematic review of current approaches. The following sections describe

1. the **research approach** used to identify the frameworks,
2. the **common frameworks** recognized from the search along with short descriptions
3. the **popularity and impact** of those tools

Afterwards, the **evaluation criteria** used to compare the frameworks are presented and clarified, along with a presentation of the tool used to compile those results.

2.1 Research Approach

To identify relevant frameworks used by the systems programming and database community, first the scope of venues and journals was defined. The list is comprised of:

- Proceedings of the VLDB Endowment (PVLDB)
- ACM Special Interest Group on Management of Data (SIGMOD)
- Conference on Innovative Data Systems Research (CIDR)
- USENIX Symposium on Operating Systems Design and Implementation (OSDI)
- USENIX Symposium on Networked Systems Design and Implementation (NSDI)
- USENIX Conference on File and Storage Technologies (FAST)

These conferences were chosen due to their high selectivity and special relevancy for storage technology research, where there is a prevalence of standardized IO benchmarks. The search was conducted broadly following the steps indicated in [1], as defined below:

1. Word search on keyword search matrix (see table 2.1)
2. Identify the frameworks used in the papers
3. Extended keyword search, this time adding the frameworks identified in the previous step

The first step suggests the use of a keyword matrix. This works by choosing appropriate keywords which refer to the topic of interest. Then for each such keyword, synonyms are used to improve the accuracy and scope of the search. It's defined below as:

	Keyword 1	Keyword 2	Keyword 3
synonyms	zipf zipfian	benchmark workload evaluation empirical dataset	synthetic generated sampled sampler

Table 2.1: Keyword search matrix

The columns are the keywords which are boolean OR'ed, whereas the rows are boolean AND'ed. This means that at least one word of each column should appear in the search.

These queries where then run for the conferences and journals listed before on an unrestricted timescale. The databases consulted are

- IEEE Xplore
- ACM Digital Library
- DBLP
- Google Scholar

2.2 Common Frameworks

On a first phase, the following frameworks were identified among the top 20 results of each conference

Frameworks
Yahoo! Cloud Serving Benchmark (YCSB)
Flexible I/O Tester (FIO)
Sysbench
OLTP-Bench [2]
DBgen

These frameworks were then searched with the same keyword matrix and their name as a last keyword. This has yielded the following results:

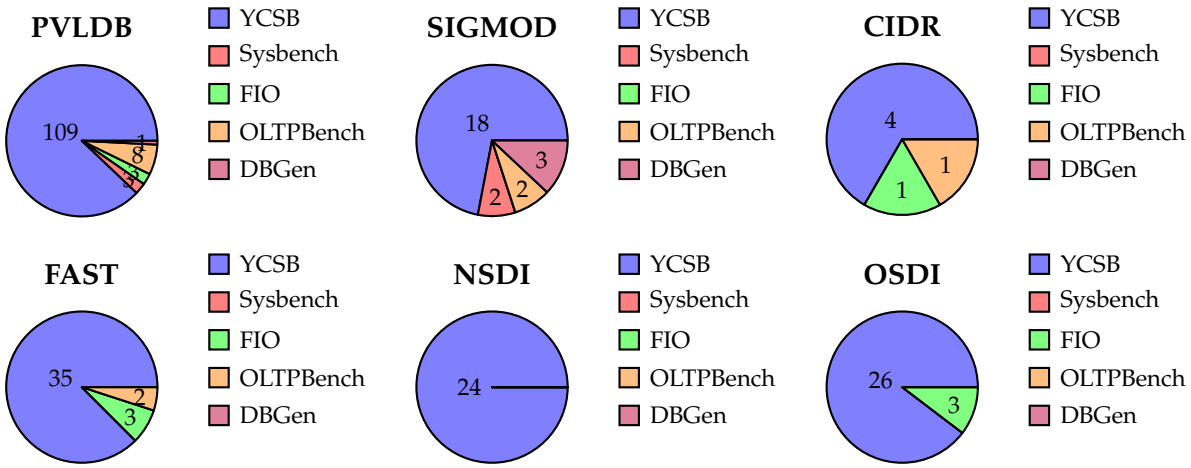


Figure 2.1: Pie charts for each conference (excluding Web Polygraph and pgBench).

The most popular framework by far is **YCSB**, then followed by **Sysbench** along with **FIO**. Other frameworks are not as prevalent, but have appeared in the search results.

2.2.1 Yahoo! Cloud Serving Benchmark

YCSB [3] is a widely used benchmarking tool for cloud databases. It was developed by *Yahoo! Research* and is now maintained by a community of developers. Its working principle is to generate workloads in a threaded environment, such that the databases can be also tested for concurrent accesses. It's also capable to loading tables with data organized according to different distributions, among which the Zipfian distribution.

To achieve this, YCSB uses a Zipfian generator which is based on the one proposed by Jim Gray [4]. One limiting factor on his original paper is that it's the generated keys are lexographically sorted, meaning the first rank (in the case of an integer, 0)

is the most frequent. To better reflect real workloads, the keys should be randomly distributed within the range. See figure 2.2 for a comparison of the two distributions.

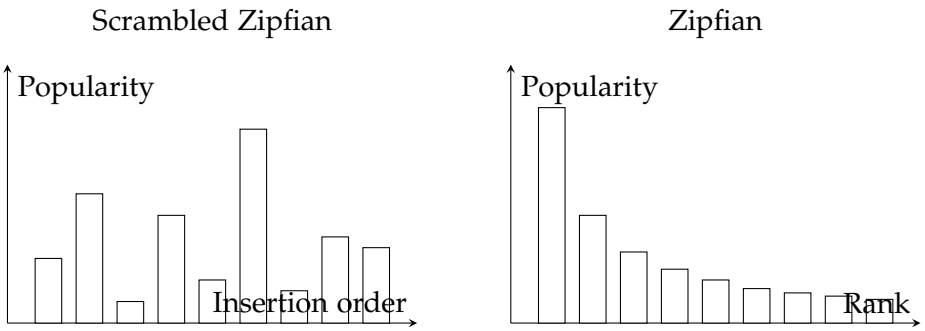


Figure 2.2: Comparison of Jim Gray's version (right) and the one employed by YCSB (left).

To go around this issue, YCSB uses a hashing algorithm to randomly spread the values in the range as well as employing an extended domain to reduce collisions [3].

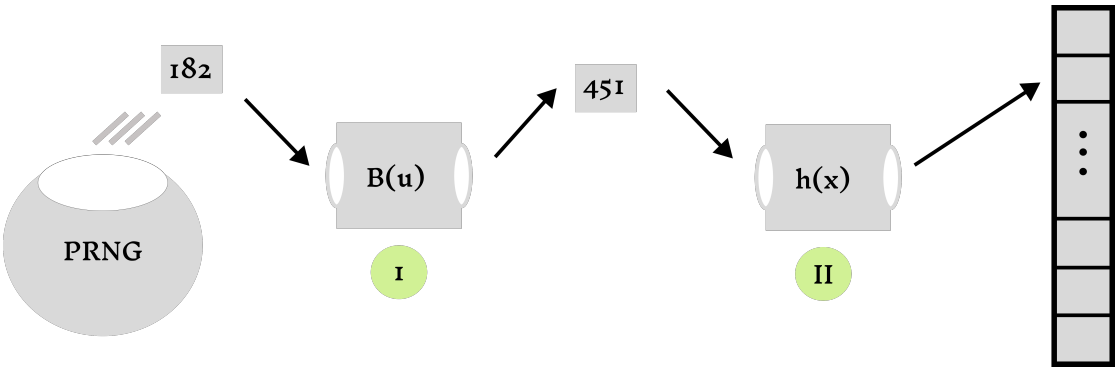


Figure 2.3: YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2)

2.2.2 Flexible I/O Tester

FIO [5] is a benchmarking tool for storage systems. Much like YCSB, its purpose is generating workloads following a certain pattern access to test a storage system. The workloads can vary in type, including sequential or random and read-only, write-only or mixed. In the case of **random I/O** , the user is able to to change the access pattern with the goal of simulating both asymetrical and more symmetrical workloads. The underlying distributions include:

- Zipfian
- Normal
- Pareto
- Uniform

The generation of a Zipfian distribution works by the same principle as YCSB, meaning:

1. Generate a zipfian variate using the algorithm proposed by Jim Gray [4]
2. Hash the value to spread it in the range

The same principle applies as in 2.3

2.2.3 Sysbench

Sysbench [6] is a benchmarking tool for databases as well as storages systems. It works in a manner very similar to YCSB, using multi-threading to strain the system with concurrent accesses. The tool is capable of generating synthetic data according to different probability distributions, among which the Zipfian distribution.

What sets it apart from the previous ones is that the variate generation uses a different algorithm. The one employed by Sysbench is based on a paper by Wolfgang Hörmann [7] and the implementation is based off the Apache Commons Math library [8], under the name *RejectionInversionZipfSampler*.

2.2.4 OLTP-Bench

The OLTP-Bench [2] strives to unify different types of benchmarks for evaluating a given Database Management System (DBMS). It favours a modular design, enabling very different benchmarks to be used under a common interface, which makes it quite extensible. However, it also has enough benchmarks built-in to be used directly, among which:

- AuctionMark [9]
- Transaction Processing Performance Council Benchmark C (TPC-C)
- YCSB

The focus will be rather layed upon the AuctionMark benchmark as it's the one excepting YCSB which generates synthetic data following a Zipfian distribution. The implementation is based on an inverse transformation sampling sped up by a binary search.

Abbreviations

TUM Technical University of Munich

PVLDB Proceedings of the VLDB Endowment

SIGMOD ACM Special Interest Group on Management of Data

CIDR Conference on Innovative Data Systems Research

OSDI USENIX Symposium on Operating Systems Design and Implementation

NSDI USENIX Symposium on Networked Systems Design and Implementation

FAST USENIX Conference on File and Storage Technologies

YCSB Yahoo! Cloud Serving Benchmark

FIO Flexible I/O Tester

DBMS Database Management System

TPC-C Transaction Processing Performance Council Benchmark C

List of Figures

1.1 Example drawing 1
1.2 Example plot 2
1.3 Example listing 2

2.1 Pie charts for each conference (excluding Web Polygraph and pgBench). 5
2.2 Comparison of Jim Gray's version (right) and the one employed by YCSB (left). 6
2.3 YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2) 6

List of Tables

1.1 Example table 1

2.1 Keyword search matrix 4

Bibliography

[1] W. M. Bramer, G. B. de Jonge, M. L. Rethlefsen, F. Mast, and J. Kleijnen, "A systematic approach to searching: An efficient and complete method to develop literature searches," *Journal of the Medical Library Association : JMLA*, vol. 106, no. 4, pp. 531–541, Oct. 2018, ISSN: 1536-5050. DOI: 10.5195/jmla.2018.283. pmid: 30271302.

[2] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, Dec. 1, 2013, ISSN: 2150-8097. DOI: 10.14778/2732240.2732246.

[3] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing*, ser. SoCC '10, New York, NY, USA: Association for Computing Machinery, Jun. 10, 2010, pp. 143–154, ISBN: 978-1-4503-0036-0. DOI: 10.1145/1807128.1807152.

[4] "Quickly generating billion-record synthetic databases | ACM SIGMOD Record." (), [Online]. Available: https://dl.acm.org/doi/10.1145/191843.191886 (visited on 11/19/2024).

[5] "1. fio - Flexible I/O tester rev. 3.38 — fio 3.38-19-gd1eb documentation." (), [Online]. Available: https://fio.readthedocs.io/en/latest/fio_doc.html (visited on 01/16/2025).

[6] A. Kopytov, *Akopytov/sysbench*, Jan. 16, 2025.

[7] W. Hörmann and G. Derflinger, "Rejection-inversion to generate variates from monotone discrete distributions," *ACM Transactions on Modeling and Computer Simulation*, vol. 6, no. 3, pp. 169–184, Jul. 1996, ISSN: 1049-3301, 1558-1195. DOI: 10.1145/235025.235029.

[8] "Math – Commons Math: The Apache Commons Mathematics Library." (), [Online]. Available: https://commons.apache.org/proper/commons-math/ (visited on 02/22/2025).

[9] "AuctionMark: An OLTP Benchmark for Shared-Nothing Database Management Systems « H-Store." (), [Online]. Available: `https://hstore.cs.brown.edu/projects/auctionmark/` (visited on 02/22/2025).